Поиск подстроки в строке

Что такое подстрока?

Пусть P это паттерн, а T - текст. Задача о точном совпадении заключается в отыскании всех вхождение паттерна P в текст T.

Строка S - упорядоченный список символов. Для любой строки S обозначим **подстроку**, как $S[i;j]$, которая начинается в i-ой и заканчивается в j-ой позициях. *Префиксом* строки S называется подстрока $S[0;i]$, а *суффиксом* - $S[i;|S|-1]$.

Собственные префикс, суффикс и подстрока строка S - не пустные и не совпадающеие с иходной строки

Наивный алгоритм

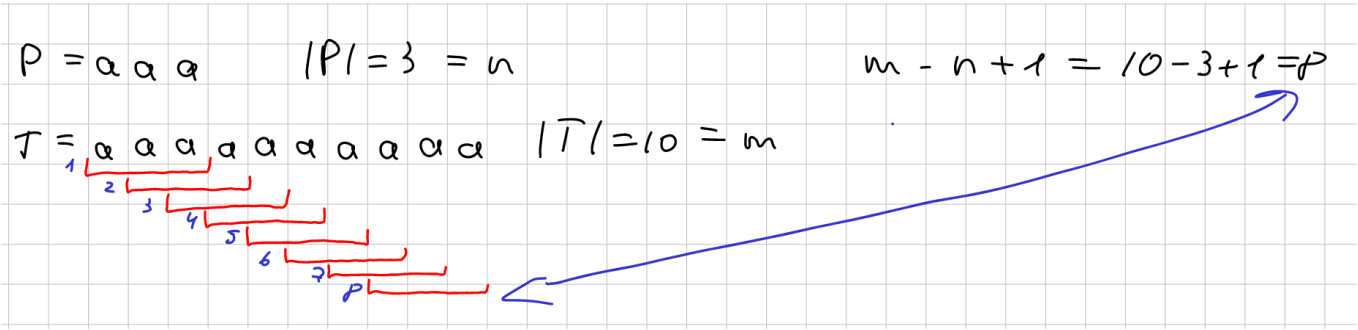
Алгоритм:

- Левый конец P ставится на одну позицию с левым концом T.
- Начинаем посимвольно сравнивать элементы слева направо, пока либо не встретится несовпадение, либо не закончится P.
  - Если P закончился, то найдено вхождение P в T.
- При любом исходе P сдвигается ровно на одну позицию вправо и алгоритм сравнения повторяется с левого конца P.
- Сравнения заканчиваются, когда правый конец P выйдет за правый конец T.

Пусть $|P| = n$, а $|T| = m$, тогда в худшем случае сложность алгоритма будет $O(n \cdot m)$

Сколько сравнений потребуется, если символы в P и T одинаковы.

Если P и T состоят из одного и того же повторяющегося символа, то P входит в текст в каждой из $m - n + 1$ позиций и метод выполняет ровно $n(m - n + 1)$ сравнений. В таком случае сложность алгоритма составляет $O(n(m - n))$.



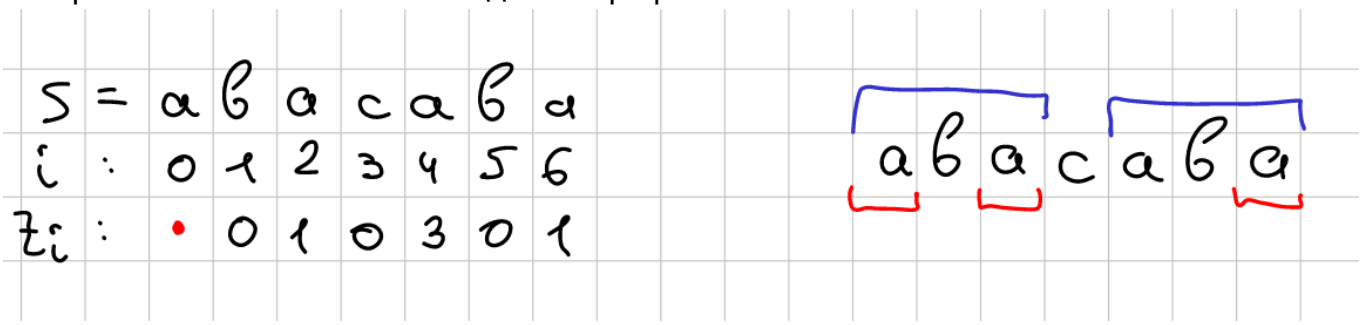
Упражнения

Ещё упражнения

Z-функция

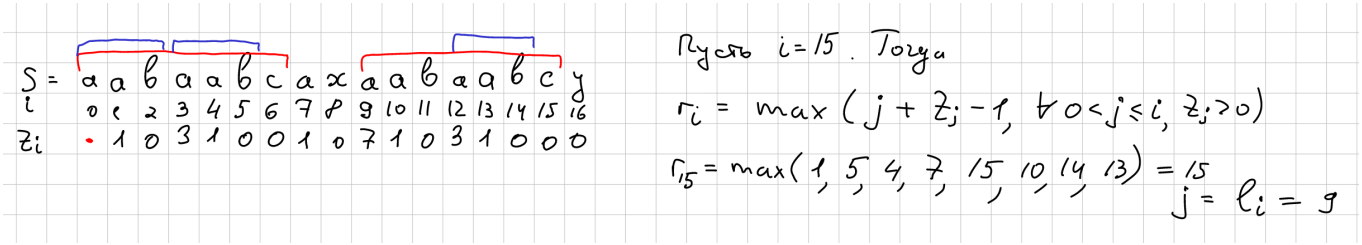
Что такое Z-функция? Как формально определяется значение $Z[i]$ для позиции i в строке? В чём интуитивный смысл Z-функции?

Для строки S и позиции $i > 0$ определим Z_i как длину наибольшей подстроки S, которая начинается в i и совпадает с префиксом S.



Что такое Z-блок? Какова длина массива Z-функции по отношению к длине строки?

Для любой позиции $i > 0$, в которой $Z_i > 0$, определеим **Z-блок** в i, как интервал, который начинается в i и заканчивается в $i + z_i - 1$.
Для любого $i > 0$ обозначим r_i - крайний правый конец Z-блоков, начинающий не позднее i-ой позиции. Т.е. r_i можно посчитать как $max(j + Z_j - 1) \quad \forall\, 0 < j \leq i,\ Z_j > 0$.
Обозначим j, на котором достигается этот максимум, через l_i. Т.е. l_i - позиция левого конца Z-блока, кончающегося в r_i. Если в r_i кончается несколько блоков, то в качестве l_i берётся любой из их левых концов.



Алгоритм поиска подстроки в строке с помощью Z-блоков.

Препроцессорный алгоритм вычисляет Z_i, r_i и l_i для каждого $i > 1$. Все значения Z сохраняются алгоритмом, в каждой итерации используются только r_{i-1} и l_{i-1}, более ранние значение l и r не требуются. Поэтому достаточно хранить значения последних

вычислений l и r. В такой реализации при нахождении нового Z-блока r возрастает до достижения конца этого Z-блока и сохраняет статус самой правой позиции.

Идея в том, чтобы использовать вычисленные значения Z_k для ускорения расчёта. Пусть $k = 121$. Тогда все значения от Z_1 до Z_{120} уже вычислены, а $r_{120} = 130$, $l_{120} = 100$. Это значит, что подстрока длины 31, начинающаяся с позиции 100, совпадает с префиксом строки S длины 31. В таком случае подстрока длины 10, начинаяющаяся в позиции 121 должна совпадать с подстрокой длины 10, начинающейся с позиции 21. Пусть $Z_{21} = 3$, тогда Z_{121} = 3

Алгоритм:
При заданных $Z_i \; \forall \; 0 < i \leq k$ и текущих l и r величина Z_k и изменения l и r вычисляются так:

- если $k > r$, то Z_k проводится последовательным сравнением до первого несовпадаения `while (k + z[k] < n && s[z[k]] == s[k + z[k]])`
  - Если новая правая граница, найденного Z-блока превышает старую (`k + z[k] - 1 > r`), то соответствующие значения обновляются так: `l = k`, `r = k + z[k] - 1`
- если $k \leq r$, то позиция k уже находится внутри ранее обнаруженного Z-блока $[l \dots r]$.
  По определению Z-блока, подстрока $S[l \dots r]$ совпадает с префиксом $S[0 \dots r - l]$. Следовательно, подстрока, начинающаяся в позиции k, должна быть идентична подстроке, начинающейся в позиции $k' = k - l$ соответствующего префикса. Обозначим за β оставшуюся часть текущего Z-блока, начиная с позиции k и до r. Её длина составляет $|\beta| = r - k + 1$. Из этого следует, что значение $Z[k]$ как минимум равно минимуму из двух величин:
  1. $Z[k']$ (значение Z-функции в соответствующей префиксной позиции).
  2. $|\beta|$ (длина оставшейся части блока, на которую мы можем гарантированно полагаться).
     В коде это условие выглядит так: `if (k <= r) z[k] = std::min(r - k + 1, z[k - l]);`
     После этого начального присваивания алгоритм пытается увеличить $Z[k]$, сравнивая символы за пределами текущего значения r, чтобы найти максимально возможное совпадение.

В каких случаях лучше применять Z-функию? Префикс-функцию?

Когда лучше Z-функция:

1. **Поиск подстроки в тексте:** Конструкция $S = P + \# + T$ (где $\#$ — уникальный символ) позволяет очень просто найти все вхождения: там, где $Z_i = |P|$, начинается вхождение паттерна.
2. **Задачи на сравнение суффиксов с префиксом:** Например, поиск самого длинного префикса строки, который также является её подстрокой в позиции i.
3. **Поиск самого длинного палиндрома:** В сочетании с алгоритмом Манакера или при анализе строки и её перевернутой копии.
4. **Сжатие строк:** Если нам нужно найти такой минимальный блок A, что строка $S = AAA...A$ (например, для $S =$"$abcabc$", блок — "abc").

Когда лучше Префикс-функция (π):

1. **Online-обработка (Потоковая передача):** Это главное преимущество. Для вычисления $\pi[i]$ нам нужно знать только символы строки до позиции i. Для Z-функции в процессе вычисления мы можем "заглядывать" далеко вперед за счет расширения r. Если текст приходит посимвольно (из сети или большого файла), KMP (префикс-функция) позволяет искать паттерн, не храня весь текст в памяти.
2. **Построение автоматов:** На основе префикс-функции легко строится конечный автомат для поиска паттерна, который переключает состояния мгновенно при чтении нового символа.
3. **Сложные задачи на суффиксы:** Когда важно знать длину наибольшего собственного префикса, который одновременно является суффиксом текущей подстроки.

Пример: Если тебе нужно найти вхождения слова в файл размером 10 ГБ, используй **префикс-функцию** (KMP), чтобы не загружать весь файл. Если тебе нужно проанализировать короткую строку на наличие внутренних повторов — **Z-функция** нагляднее.

Что случится с Z-функцией, если добавить специальный символ в начало или конец?

1. Добавление в середину (Случай $S = P + \# + T$)

Пусть $\#$ — символ, который **не встречается** ни в P, ни в T.

- **Зачем это нужно:** Разделитель гарантирует, что значение Z_i для любого i, относящегося к тексту T, **не превысит** длину паттерна P.
- **Результат:** Когда алгоритм дойдет до символа $\#$, сравнение сразу прекратится (так как $\#$ уникален). Это "сбрасывает" Z-блок и не дает значению Z "перетекать" из паттерна в текст.

- **Пример:** $P = aba$, $T = ababa$.
  Строка: $aba\#ababa$
  Z-массив: $[\cdot, 0, 1, 0, 3, 0, 3, 0, 1]$
  Здесь $Z_i = 3$ в позициях текста четко указывают на вхождения.

2. Добавление в начало

Если добавить уникальный символ в самое начало (например, $\#S$):

- Z_0 станет равно $|S| + 1$.
- Все остальные Z_i станут равны 0, так как ни один суффикс строки не начнется с этого уникального символа $\$$, а значит, не совпадет с префиксом.
- **Это бесполезно** для поиска, но используется в некоторых теоретических доказательствах.

3. Добавление в конец

Если добавить уникальный символ в конец ($S + \#$):

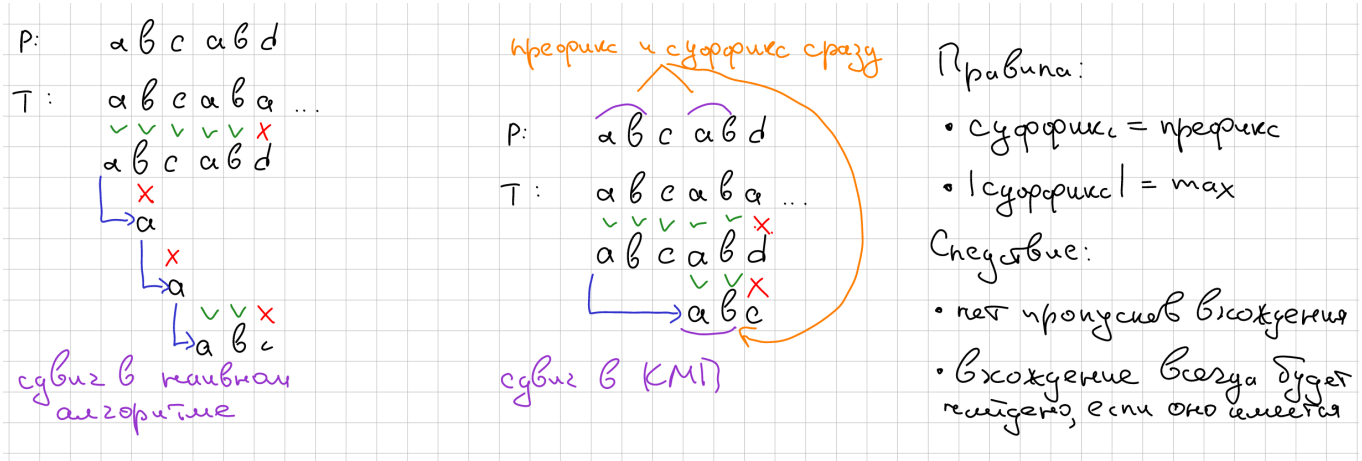
- Это никак не изменит уже вычисленные значения Z_i для основной строки.
- Это полезно, чтобы гарантировать, что последний Z-блок будет корректно закрыт, или при построении суффиксных структур, чтобы каждый суффикс был уникальным (не являлся префиксом другого суффикса).

Упражнение

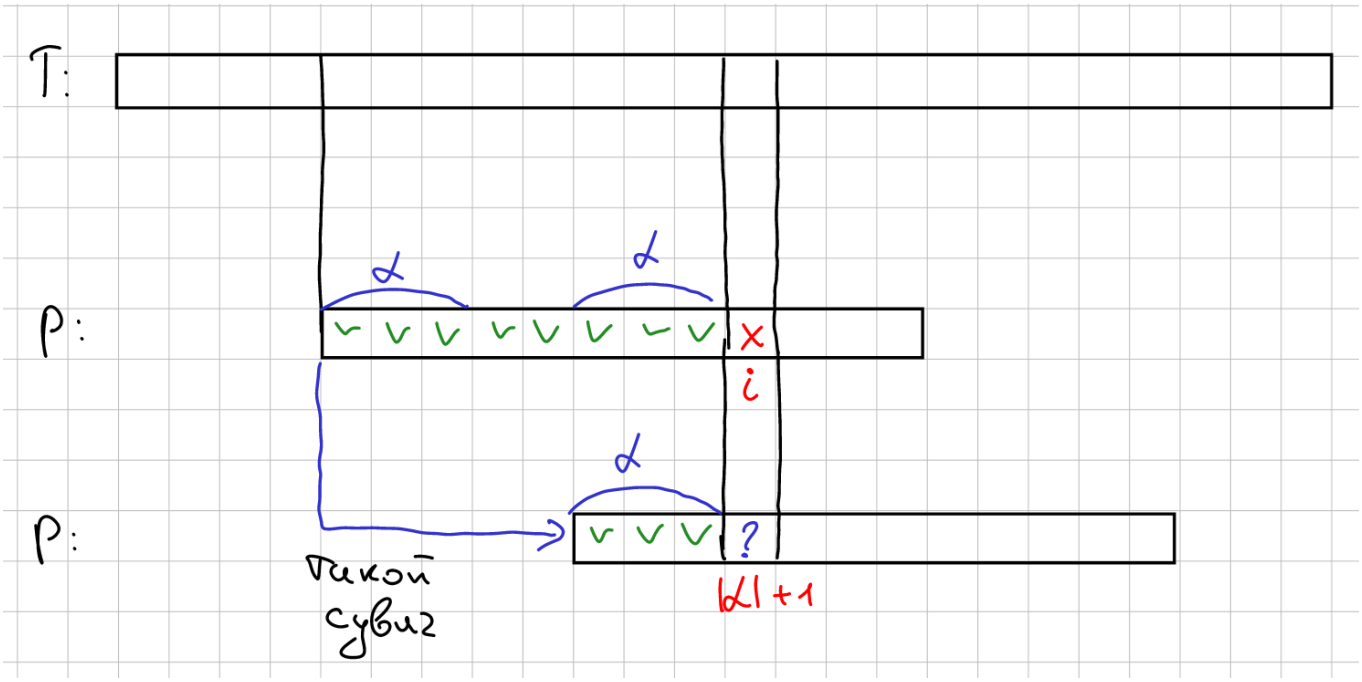
22. В п. 1.5 мы показали, что все вхождения P в T можно найти за линейное время вычислением значений Z для строки $S = P\$T$. Объясните, как нужно изменить этот метод, если мы используем $S = PT$, т.е. не ставим разделителя между строками P и T. Покажите теперь, как найти все вхождения P в T за линейное время, используя $S = PT$, но со значениями sp вместо Z. (Это не так просто, как кажется на первый взгляд.)

КМП

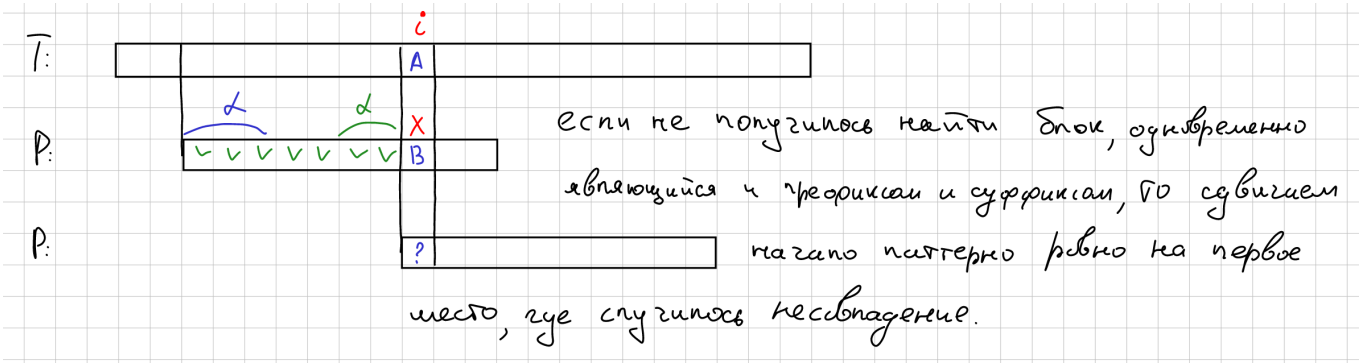
Правило сдвига



В общем виде:



Если не получилось найти блок, одновременно являющийся и префиксом, и суффиксом, то сдвигаем начало паттерна ровно на первое место, где случилось несовпадение



Препроцессинг

SP-функция

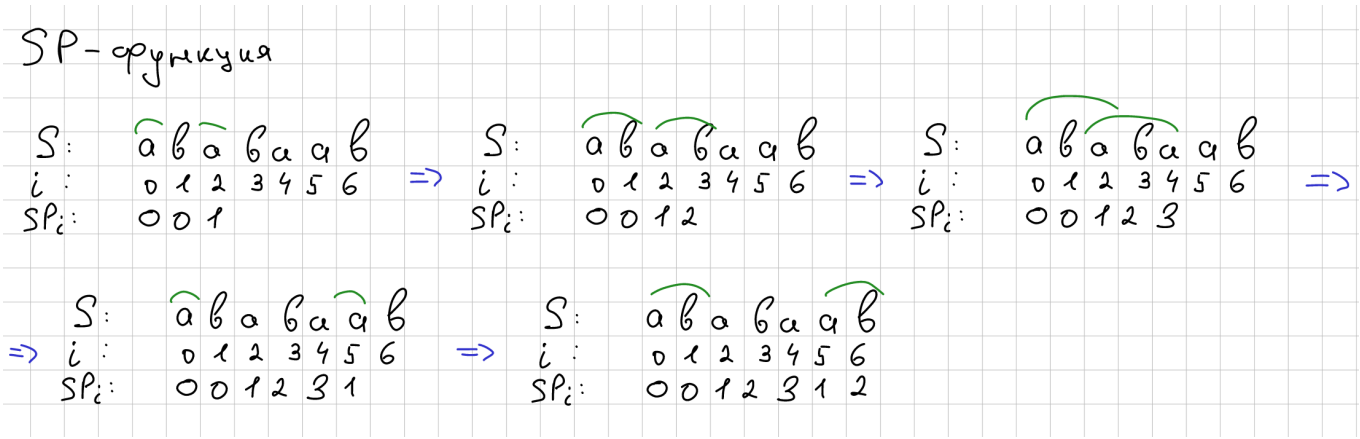
Определение: $sp[i]$ — это длина максимального собственного префикса строки $S[0 \dots i]$, который одновременно является её суффиксом.

Алгоритм (вычисляем массив sp для строки S длины n):

1. Базовый случай: $sp[0] = 0$ (у строки из одного символа нет собственного префикса).
2. Заводим переменную `j = 0`. Она будет хранить длину текущего совпавшего префикса (и одновременно индекс следующего символа, с которым мы будем сравнивать $S[i]$).
3. Проходим циклом `for` по i от 1 до $n-1$:
   - **Пока** `j > 0` и $S[i] \neq S[j]$, мы откатываем `j` назад: `j = sp[j - 1]`. (Пытаемся найти префикс поменьше).
   - **Если** $S[i] == S[j]$, то мы можем удлинить префикс: `j++`.
   - Записываем результат: `sp[i] = j`.

Сложность: $O(n)$ по времени и памяти.

Пример вычисления SP-функции



i	**Символ** $S[i]$	**Текущее** j	**Сравнение** $S[i]$ **и** $S[j]$	**Действие / Откат**	**Результат** $sp[i]$
0	**a**	0	-	Базовый случай	**0**
1	**b**	0	'b' $\neq$ 'a' ($S[0]$)	j уже 0, совпадений нет	**0**
2	**a**	0	'a' $==$ 'a' ($S[0]$)	Совпало! $j \to 1$	**1**
3	**b**	1	'b' $==$ 'b' ($S[1]$)	Совпало! $j \to 2$	**2**
4	**a**	2	'a' $==$ 'a' ($S[2]$)	Совпало! $j \to 3$	**3**
5	**a**	3	'a' $\neq$ 'b' ($S[3]$)	Не совпало! Откат: $j = sp[2] = 1$. Сравниваем 'a' с $S[1]$	**1**

i	**Символ** $S[i]$	**Текущее** j	**Сравнение** $S[i]$ **и** $S[j]$	**Действие / Откат**	**Результат** $sp[i]$
				('b') - нет. Откат: $j = sp[0] = 0$. Сравниваем 'a' с $S[0]$ ('a') - да! $j \to 1$	
6	**b**	1	'b' $==$ 'b' ($S[1]$)	Совпало! $j \to 2$	**2**

Итог: $sp = [0, 0, 1, 2, 3, 1, 2]$

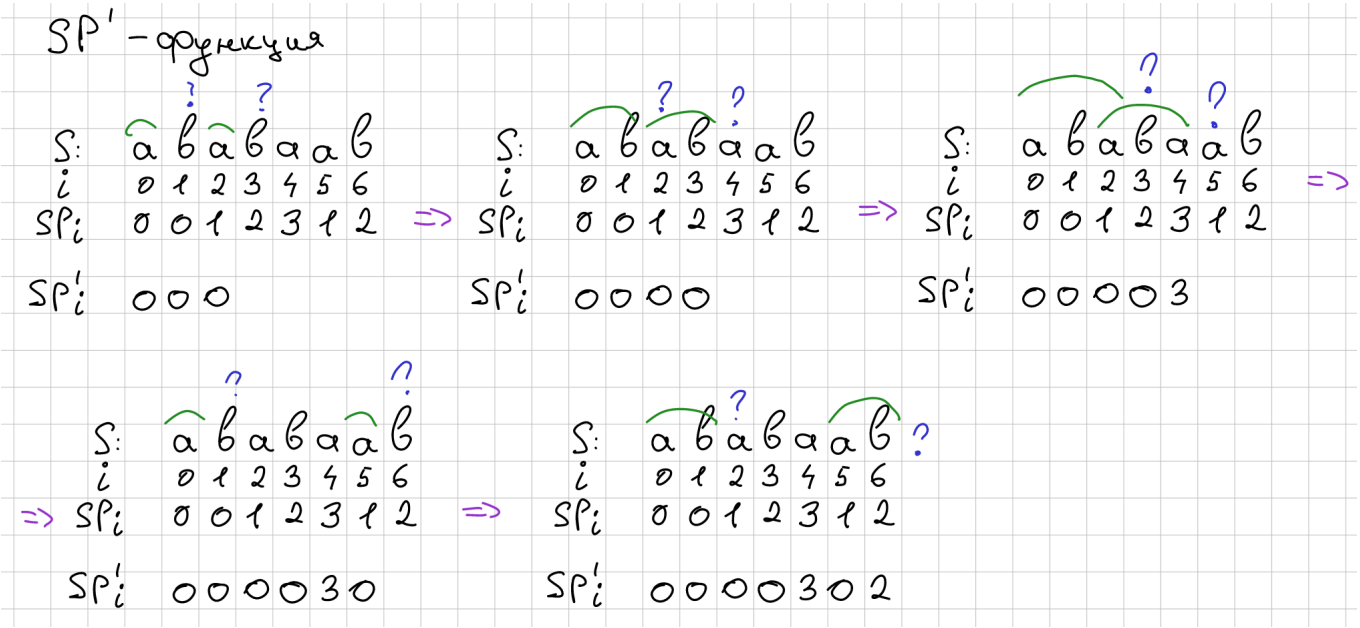
SP'-функция

Зачем нужна: При обычном КМП, если произойдет несовпадение текста с символом $S[i+1]$, мы сделаем сдвиг и будем сравнивать текст с $S[sp[i]]$. Но если $S[sp[i]] == S[i+1]$, то новое сравнение **гарантированно приведет к ошибке**, и мы потратим шаг впустую.
Сильная SP-функция ($sp'[i]$) — это длина такого максимального префикса-суффикса, за которым следует символ, **отличный** от $S[i+1]$. То есть: $S[sp'[i]] \neq S[i+1]$.

Алгоритм (вычисляется за $O(n)$ на основе обычного sp):

1. Сначала вычисляем обычный массив sp.
2. Создаем массив sp'. (Для последнего элемента $sp'[n-1] = sp[n-1]$, так как за ним нет символа).
3. Проходим по i от 0 до $n-2$:
   - Если $sp[i] == 0$, то $sp'[i] = 0$.
   - Иначе смотрим на символы, следующие за префиксом и суффиксом:
     - Если $S[sp[i]] \neq S[i+1]$, то нас всё устраивает: $sp'[i] = sp[i]$.
     - Если $S[sp[i]] == S[i+1]$, сдвиг по $sp[i]$ бесполезен. Мы берем "сильное" значение от того префикса, в который бы перешли: $sp'[i] = sp'[sp[i]-1]$.

Пример вычисления SP'-функции



i	$S[i]$	$sp[i]$	**След. символ за префиксом ($S[sp[i]]$)**	**След. символ в паттерне ($S[i+1]$)**	**Сравнение**	**Результат $sp'[i]$**
0	**a**	0	$S[0] =$ **a**	$S[1] =$ **b**	**a** $\neq$ **b**	**0**
1	**b**	0	$S[0] =$ **a**	$S[2] =$ **a**	**a** $==$ **a**	**0**
2	**a**	1	$S[1] =$ **b**	$S[3] =$ **b**	**b** $==$ **b**	**0** (Берем $sp'[1-1] = sp'[0]$)
3	**b**	2	$S[2] =$ **a**	$S[4] =$ **a**	**a** $==$ **a**	**0** (Берем $sp'[2-1] = sp'[1]$)
4	**a**	3	$S[3] =$ **b**	$S[5] =$ **a**	**b** $\neq$ **a**	**3** (Хороший сдвиг, символы разные)
5	**a**	1	$S[1] =$ **b**	$S[6] =$ **b**	**b** $==$ **b**	**0** (Берем $sp'[1-1]$)
6	**b**	2	—	—	Последний символ	**2** (Обычно оставляем sp)

Итог: $sp' = [0, 0, 0, 0, 3, 0, 2]$

Как используется $sp[i-1]$, когда при вычислении $sp[i]$ возникает несовпадение?

Пусть мы вычислили $sp[i-1] = j$. Это значит, что префикс длины j совпадает с суффиксом перед текущим символом. Мы пытаемся сравнить $S[i]$ и $S[j]$.

Если возникает несовпадение ($S[i] \neq S[j]$), это значит, что мы не можем удлинить префикс длины j. Нам нужно найти **следующий по длине** префикс, который также является суффиксом для строки $S[0 \dots i-1]$.

Где его искать? Поскольку префикс $S[0 \dots j-1]$ уже идентичен суффиксу перед позицией i, нам достаточно найти максимальный префикс-суффикс **внутри самого** $S[0 \dots j-1]$.
А его длина уже вычислена и хранится в $sp[j-1]$!
Поэтому мы просто делаем присваивание `j = sp[j - 1]` и повторяем сравнение. Это избавляет нас от необходимости откатываться по строке назад и заново посимвольно сравнивать элементы.

Пример

Пусть мы находимся на шаге i. До этого мы уже нашли совпадение длины $j = 6$.

```
Индексы:  0 1 2 3 4 5 6      ...   i-6 i-5 i-4 i-3 i-2 i-1   i
Строка S: a b a c a b [X]    ...    a   b   a   c   a   b   [Y]
```

Мы сравниваем $S[i]$ (пусть это **'Y'**) и $S[6]$ (пусть это **'X'**).
Они **не совпали**. Что делать?

1. Мы точно знаем, что суффикс перед i совпадает с префиксом длины 6: `abacab` == `abacab`.
2. Мы не можем продолжить префикс длины 6. Нам нужен префикс поменьше.
3. Где его искать? Мы ищем наибольшее начало строки, которое совпадает с концом блока `abacab`.
4. Но ведь конец левого блока `abacab` абсолютно идентичен концу правого блока `abacab`!
5. Значит, нам просто нужно посмотреть значение $sp[5]$ (это $sp[j-1]$) для самого левого префикса.

Смотрим на левый префикс (индексы 0-5):

```
[a b] a c [a b]
 0 1       4 5
```

$sp[5] = 2$ (префикс "ab" равен суффиксу "ab").
Значит, мы можем мгновенно перескочить на $j = 2$ и пытаться сравнивать $S[i]$ уже с $S[2]$ (с символом 'a').

Почему суммарное число сравнений при вычислении префикс-функции линейно от длины строки?

Хотя внутри цикла `for` (от 1 до $n-1$) есть цикл `while`, алгоритм работает за $O(n)$. Для доказательства используется метод амортизационного анализа.

Рассмотрим переменную `j` (длину текущего префикса-суффикса):

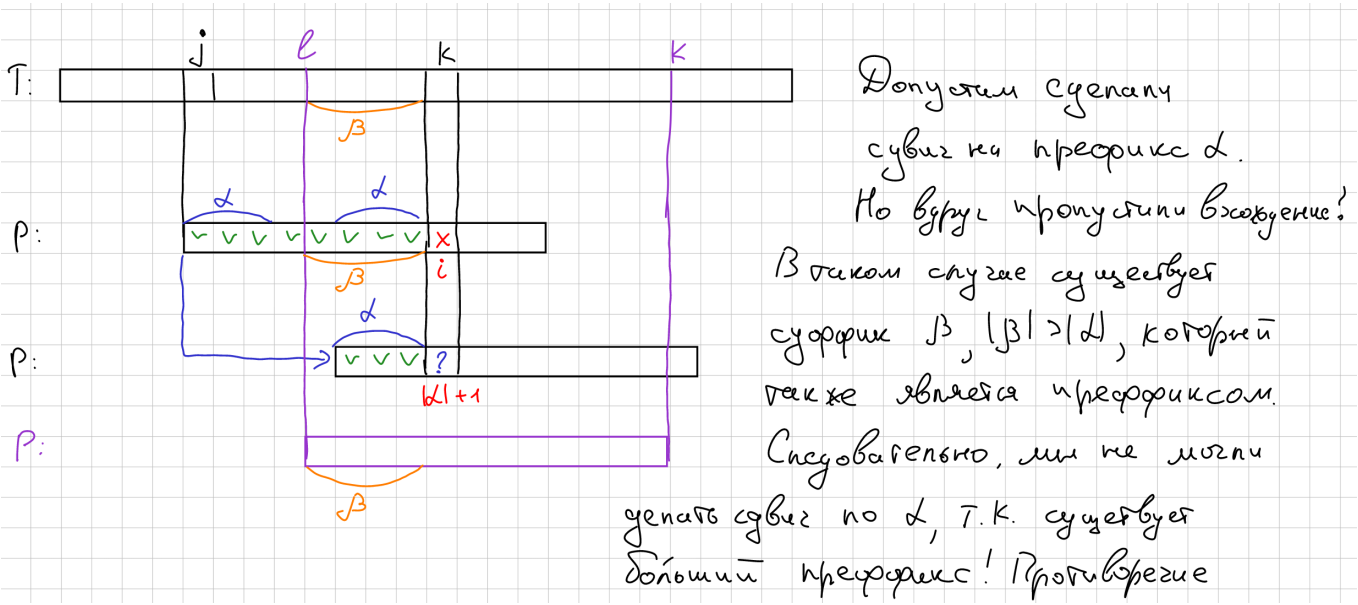
1. Изначально `j = 0`.
2. В каждой итерации цикла `for` значение `j` может увеличиться **максимум на 1** (в строке `j++`). Значит, за весь алгоритм `j` увеличится не более чем $n-1$ раз.
3. В цикле `while` при несовпадении мы делаем `j = sp[j - 1]`. Так как $sp[k] < k+1$, значение `j` строго **уменьшается** как минимум на 1.
4. Поскольку `j` не может стать отрицательным, суммарное количество уменьшений `j` (итераций цикла `while`) не может превышать суммарного количества увеличений `j`.
5. Следовательно, общее число итераций цикла `while` за всё время работы алгоритма ограничено числом n. Общее время работы строго пропорционально длине строки: $O(n)$.

Чем Z-функция отличается от SP-функции?

ЭТО ЧИСТО МОЁ МНЕНИЕ!!!
SP-функция считает количество совпадающих с паттерном элементов перед текущим
Z-функция считает количество совпадающих с паттерном элементов после текущего

Доказательство корректности алгоритма

Доказательство, что нет пропусков вхождений:



Допустим сделали сдвиг на префикс α. Но оказалось, что было прощуено вхождение. Тогда существует суффикс $\beta : |\beta| > |\alpha|$, который также является префиксом. *Следовательно*, нельзя было сдвигать по α, т.к. существует префикс большей длины. *Противоречие!*

Максимальное количество сравнений в КМП? Доказательство.

Теорема: В КМП количество сравнений не превышает $2m$.
Доказательство: Алгоритм КМП делится на фазы сравнения и фазы сдвига. Сравнения происходят после каждого сдвига и начинаются с последнего сравниваемого символа в T. Получается в каждой фазе сравнения будет не более m сравнений. А суммарное количество будет не более $m + s$ сравнений, где s - количество сдвигов. Очевидно, что в наихудшем случае будет $s = m$ сдигов, *следовательно*, общее количество сравнений никогда не превысит $2m$.

В чём преимущество КМП над Z-функцией? Когда лучше использовать z-функцию? префикс-функцию?

Поиск строк в реальном времени

Упражнения

БМ

Апостолико-Джанкарло

Ахо-Корасик

Код

Наивный алгоритм

```
std::vector<int> naiveSearch(const std::string& T, const std::string& P) {
    int m = T.size();
    int n = P.size();
    std::vector<int> idxs;

    if (n == 0) return idxs;

    for (int i = 0; i <= m - n; ++i) {
        int j = 0;
        while (j < n && T[i + j] == P[j]) {
```

```
            ++j;
        }
        if (j == n) {
            idxs.emplace_back(i);
        }
    }
    return idxs;
}
```

Z-фукнция за линию

```
std::vector<int> zFunction(const std::string& s) {
    int n = s.size();
    std::vector<int> z(n, 0);
    int left = 0, right = 0;


    for (int i = 1; i < n; ++i) {
        if (i <= right) {
            z[i] = std::min(right - i + 1, z[i - left]);
        }

        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            ++z[i];
        }

        if (i + z[i] - 1 > right) {
            left = i;
            right = i + z[i] - 1;
        }
    }
    return z;
}
```

SP

```
std::vector<int> SP(const std::string& s) {
    int n = s.length();
    std::vector<int> sp(n, 0);

    for (int i = 1; i < n; i++) {
        int j = sp[i - 1]; // Берем длину предыдущего префикса-суффикса

        // Пока есть несовпадение и мы не дошли до начала строки,
```

```
        // откатываемся к предыдущему максимальному префиксу
        while (j > 0 && s[i] != s[j]) {
            j = sp[j - 1];
        }

        // Если символы совпали, увеличиваем длину префикса
        if (s[i] == s[j]) {
            j++;
        }

        sp[i] = j;
    }

    return sp;
}
```

SP'

```
std::vector<int> strongSP(const std::string& s) {
    int n = s.length();
    // 1. Сначала вычисляем обычную SP-функцию
    std::vector<int> sp = SP(s);
    std::vector<int> sp_prime(n, 0);

    // 2. Строим сильную SP'-функцию
    for (int i = 0; i < n; i++) {
        // Для последнего элемента оставляем как есть, т.к. за ним нет
символа
        if (i == n - 1) {
            sp_prime[i] = sp[i];
            continue;
        }

        int len = sp[i];
        if (len == 0) {
            sp_prime[i] = 0;
        } else {
            // Если символ за префиксом равен символу за суффиксом,
            // сдвиг бесполезен, берем значение от предыдущего префикса
            if (s[len] == s[i + 1]) {
                sp_prime[i] = sp_prime[len - 1];
            } else {
                // Если символы разные, сдвиг хороший
                sp_prime[i] = len;
            }
```

```
        }
    }

    return sp_prime;
}
```

SP' из Z-функции

```
std::vector<int> SPfromZ(const std::vector<int>& z) {
    int n = z.size();
    std::vector<int> sp(n, 0);

    // 1. Проход справа налево (записываем концы Z-блоков)
    for (int j = n - 1; j > 0; j--) {
        if (z[j] > 0) {
            int i = j + z[j] - 1; // Конец совпадения
            // Перезаписываем, так как меньший j дает больший префикс
            sp[i] = z[j];
        }
    }

    // 2. Проход слева направо (заполняем "дырки")
    // Если в позиции i есть префикс длины L,
    // то в позиции i-1 точно есть префикс длины L-1.
    for (int i = n - 2; i >= 0; i--) {
        if (sp[i + 1] > 0) {
            sp[i] = std::max(sp[i], sp[i + 1] - 1);
        }
    }

    return sp;
}
```

КМП

```
std::vector<int> KMPSearch(const std::string& text, const std::string&
pattern) {
    std::vector<int> occurrences;
    int m = text.length();
    int n = pattern.length();

    if (n == 0) return occurrences;

    // Вычисляем префикс-функцию (можно заменить на strongSP или SPfromZ)
```

```
    std::vector<int> sp = SP(pattern);

    int j = 0; // Индекс в паттерне (длина совпавшего префикса)

    // Проходим по тексту (i - индекс в тексте)
    for (int i = 0; i < m; i++) {
        // Если несовпадение, откатываемся по SP-массиву
        while (j > 0 && text[i] != pattern[j]) {
            j = sp[j - 1];
        }

        // Если символы совпали, идем дальше по паттерну
        if (text[i] == pattern[j]) {
            j++;
        }

        // Если весь паттерн совпал
        if (j == n) {
            occurrences.push_back(i - n + 1); // Записываем начало вхождения
            j = sp[n - 1]; // Готовимся искать следующее вхождение
        }
    }

    return occurrences;
}
```

Правило плохого символа в БМ

Trie